

PROJECT DOCUMENTATION

AXI4 Memory Verification Project

UVM Testbench - Code Architecture Documentation



PREPARED BY

Youssef Emad Ali

ID: 479

AXI4 | UVM | SystemVerilog

```

1 interface mem_if #(
2     parameter DATA_WIDTH = 32,
3     parameter ADDR_WIDTH = 10,    // For 1024 locations
4     parameter DEPTH = 1024
5 );
6     logic                clk;
7     logic                rst_n;
8     logic                mem_en;
9     logic                mem_we;
10    logic [ADDR_WIDTH-1:0] mem_addr;
11    logic [DATA_WIDTH-1:0] mem_wdata;
12    logic [DATA_WIDTH-1:0] mem_rdata;
13
14 endinterface
15

```

MEMORY INTERFACE Memory interface

```

1 import axi4_assertions_pkg::*;
2 interface axi4_if #(
3     parameter DATA_WIDTH = 32,
4     parameter ADDR_WIDTH = 16,
5     parameter MEMORY_DEPTH = 1024
6 );
7
8     logic                ACLK;
9     logic                ARESETn;
10
11    logic [ADDR_WIDTH-1:0] AWADDR;
12    logic [7:0]           AWLEN;
13    logic [2:0]           AWSIZE;
14    logic                AWVALID;
15    logic                AWREADY;
16
17    logic [DATA_WIDTH-1:0] WDATA;
18    logic                WVALID;
19    logic                WLAST;
20    logic                WREADY;
21
22    logic [1:0]           BRESP;
23    logic                BVALID;
24    logic                BREADY;
25
26    logic [ADDR_WIDTH-1:0] ARADDR;
27    logic [7:0]           ARLEN;
28    logic [2:0]           ARSIZE;
29    logic                ARVALID;
30    logic                ARREADY;
31
32    logic [DATA_WIDTH-1:0] RDATA;
33    logic [1:0]           RRESP;
34    logic                RVALID;
35    logic                RLAST;
36    logic                RREADY;
37

```

AXI INTERFACE - PART 1 AXI interface

```

39  a_reset : assert property (
40      p_reset(
41          ACLK, ARESETn,
42          AWREADY, WREADY, BVALID,
43          ARREADY, RVALID, RLAST
44      )
45  );
46  a_w_stable : assert property (
47      w_stable(
48          ACLK, ARESETn,
49          WVALID, WREADY,
50          WDATA, WLAST
51      )
52  );
53  a_bresp_valid : assert property (
54      bresp_valid(
55          ACLK, ARESETn,
56          BVALID, BRESP
57      )
58  );
59  a_rresp_valid : assert property (
60      rresp_valid(
61          ACLK, ARESETn,
62          RVALID, RRESP
63      )
64  );
65  a_no_unknown : assert property (
66      unknown_outputs(
67          ACLK,
68          ARESETn,
69          {AWREADY, WREADY, BRESP, BVALID, ARREADY, RDATA, RRESP, RVALID, RLAST}
70      )
71  );
72  w_handshake : cover property (
73      @(posedge ACLK) disable iff (!ARESETn)
74      WVALID && WREADY
75  );
76  endinterface
77

```

AXI INTERFACE - PART 2 AXI interface

```

1  package axi4_assertions_pkg;
2
3      property p_reset(
4          clk, rst_n,
5          awready, wready, bvalid,
6          arready, rvalid, rlast
7      );
8      @(posedge clk)
9      !rst_n |-> (
10          awready &&
11          !wready &&
12          !bvalid &&
13          arready &&
14          !rvalid &&
15          !rlast
16      );
17  endproperty
18
19  property w_stable(clk, rst_n, wvalid, wready, wdata, wlast);
20      @(posedge clk) disable iff (!rst_n)
21      (wvalid && !wready)
22      |>
23      (wvalid &&
24       $stable(wdata) &&
25       $stable(wlast));
26  endproperty
27
28  property bresp_valid(clk, rst_n, bvalid, bresp);
29      @(posedge clk) disable iff (!rst_n)
30      bvalid |->
31      ((bresp == 2'b00) || (bresp == 2'b10));
32  endproperty
33
34  property rresp_valid(clk, rst_n, rvalid, rresp);
35      @(posedge clk) disable iff (!rst_n)
36      rvalid |->
37      ((rresp == 2'b00) || (rresp == 2'b10));
38  endproperty
39
40  property unknown_outputs(clk, rst_n, outputs);
41      @(posedge clk) disable iff (!rst_n)
42      !$isunknown(outputs);
43  endproperty
44
45  endpackage

```

AXI ASSERTIONS Protocol assertions

```

1  package mem_transaction_pkg;
2
3  import uvm_pkg::*;
4  `include "uvm_macros.svh"
5
6  class mem_transaction extends uvm_sequence_item;
7
8      logic      rst_n;
9      logic      mem_en;
10     logic      mem_we;
11     logic [9:0] mem_addr;
12     logic [31:0] mem_wdata;
13     logic [31:0] mem_rdata;
14
15     `uvm_object_utils_begin(mem_transaction)
16         `uvm_field_int(rst_n,      UVM_DEFAULT)
17         `uvm_field_int(mem_en,      UVM_DEFAULT)
18         `uvm_field_int(mem_we,      UVM_DEFAULT)
19         `uvm_field_int(mem_addr,    UVM_DEFAULT)
20         `uvm_field_int(mem_wdata,    UVM_DEFAULT)
21         `uvm_field_int(mem_rdata,    UVM_DEFAULT)
22     `uvm_object_utils_end
23
24     function new(string name = "mem_transaction");
25         super.new(name);
26     endfunction
27
28 endclass
29 endpackage
30

```

MEMORY TRANSACTION Memory transaction

```

axi4_transaction.sv > Verilog, SV and UVM code editor > {} axi4_transaction_pkg
1  package axi4_transaction_pkg;
2  `include "uvm_macros.svh"
3  import uvm_pkg::*;
4
5  typedef enum bit {
6      WRITE_OP,
7      READ_OP,
8  } operation_e;
9
10 class axi4_transaction #(
11     parameter DATA_WIDTH = 32,
12     parameter ADDR_WIDTH = 16,
13     parameter MEMORY_DEPTH = 1024
14 ) extends uvm_sequence_item;
15
16     bit          ACLK;
17     bit          ARESETn;
18
19     operation_e  operation;
20
21     rand logic [ADDR_WIDTH-1:0] AWADDR;
22     rand logic [7:0]            AWLEN;
23     rand logic [2:0]            AWSIZE;
24     logic         AWVALID;
25     logic         AWREADY;
26
27     rand logic [DATA_WIDTH-1:0] WDATA[];
28     logic         WVALID;
29     logic         WLAST;
30     logic         WREADY;
31
32     logic [1:0]      BRESP;
33     logic            BVALID;
34     logic            BREADY;
35
36     rand logic [ADDR_WIDTH-1:0] ARADDR;
37     rand logic [7:0]            ARLEN;
38     rand logic [2:0]            ARSIZE;
39     logic         ARVALID;
40     logic         ARREADY;
41
42     logic [DATA_WIDTH-1:0] RDATA[];
43     logic [1:0]            RRESP;
44     logic                 RVALID;
45     logic                 RLAST;
46     logic                 RREADY;
47

```

AXI TRANSACTION - PART 1 Transaction fields

```

48     constraint c0 {ARSIZE == 3'd2;AWSIZE == 3'd2;}
49
50     constraint addr_c {
51         AWADDR inside {[16'h0000:16'h0FFC]};
52         ARADDR inside {[16'h0000:16'h0FFC]};
53         AWADDR[1:0] == 2'b00;
54         ARADDR[1:0] == 2'b00;
55     }
56
57     constraint burst_c {
58         (int'(AWADDR) + ((int'(AWLEN) + 1) * 4) - 1)
59             <= 16'h0FFF;
60
61         (int'(ARADDR) + ((int'(ARLEN) + 1) * 4) - 1)
62             <= 16'h0FFF;
63     }
64
65     constraint wdata_size_c { WDATA.size() == AWLEN + 1;}

```

AXI TRANSACTION - CONSTRAINTS Constraints

```

67     `uvm_object_utils_begin(axi4_transaction)
68         `uvm_field_int(ACLK, UVM_DEFAULT)
69         `uvm_field_int(ARESETn, UVM_DEFAULT)
70         `uvm_field_int(AWADDR, UVM_DEFAULT)
71         `uvm_field_int(AWLEN, UVM_DEFAULT)
72         `uvm_field_int(AWSIZE, UVM_DEFAULT)
73         `uvm_field_int(AWVALID, UVM_DEFAULT)
74         `uvm_field_int(AWREAD, UVM_DEFAULT)
75         `uvm_field_array_int(WDATA, UVM_DEFAULT)
76         `uvm_field_int(WVALID, UVM_DEFAULT)
77         `uvm_field_int(WLAST, UVM_DEFAULT)
78         `uvm_field_int(WREAD, UVM_DEFAULT)
79         `uvm_field_int(BRESP, UVM_DEFAULT)
80         `uvm_field_int(BVALID, UVM_DEFAULT)
81         `uvm_field_int(BREADY, UVM_DEFAULT)
82         `uvm_field_int(ARADDR, UVM_DEFAULT)
83         `uvm_field_int(ARLEN, UVM_DEFAULT)
84         `uvm_field_int(ARSIZE, UVM_DEFAULT)
85         `uvm_field_int(ARVALID, UVM_DEFAULT)
86         `uvm_field_int(ARREADY, UVM_DEFAULT)
87         `uvm_field_array_int(RDATA, UVM_DEFAULT)
88         `uvm_field_int(RRESP, UVM_DEFAULT)
89         `uvm_field_int(RVALID, UVM_DEFAULT)
90         `uvm_field_int(RLAST, UVM_DEFAULT)
91         `uvm_field_int(RREAD, UVM_DEFAULT)
92     `uvm_object_utils_end
93
94     function new (string name = "axi4_transaction");
95         super.new(name);
96     endfunction
97 endclass
98 endpackage
99

```

AXI TRANSACTION - PART 3 Transaction utilities

```

axi4_sequence.sv > Verilog, SV and UVM code editor > { } axi4_sequence_pkg > axi4_sequence > body
1  package axi4_sequence_pkg;
2  `include "uvm_macros.svh"
3  import uvm_pkg::*;
4  import axi4_transaction_pkg::*;
5
6  class axi4_sequence extends uvm_sequence #(axi4_transaction);
7
8      `uvm_object_utils(axi4_sequence)
9
10     function new (string name = "axi4_sequence");
11         super.new(name);
12     endfunction
13
14     task body();
15         repeat (2000) begin
16             req = axi4_transaction #(.DATA_WIDTH(32),
17                                     .ADDR_WIDTH (16),
18                                     .MEMORY_DEPTH (1024))
19                                     ::type_id::create("req");
20             start_item(req);
21             assert(req.randomize())
22             finish_item(req);
23         end
24     endtask
25 endclass
26 endpackage
27

```

SEQUENCE AXI sequence

```

axi4_sequencer.sv > Verilog, SV and UVM code editor > { } axi4_sequencer_pkg
1  package axi4_sequencer_pkg;
2  `include "uvm_macros.svh"
3  import uvm_pkg::*;
4  import axi4_transaction_pkg::*;
5
6  class axi4_sequencer extends uvm_sequencer #(axi4_transaction);
7
8      `uvm_component_utils(axi4_sequencer)
9
10     function new (string name = "axi4_sequencer", uvm_component parent);
11         super.new(name,parent);
12     endfunction
13
14     function void build_phase( uvm_phase phase);
15         super.build_phase(phase);
16         `uvm_info(get_type_name(), "Inside Sequencer [Build]", UVM_LOW)
17     endfunction
18 endclass
19 endpackage
20

```

SEQUENCER AXI sequencer

```

1  package axi4_driver_pkg;
7  class axi4_driver extends uvm_driver #(axi4_transaction);
8
9      `uvm_component_utils(axi4_driver)
10     virtual axi4_if axi_if;
11     common_cfg m_cfg;
12     function new (string name = "axi4_driver", uvm_component parent);
13         super.new(name,parent);
14     endfunction
15
16     function void build_phase( uvm_phase phase);
17         super.build_phase(phase);
18         if (!uvm_config_db #(virtual axi4_if)::get(this,"", "vif",axi_if))
19             `uvm_fatal(get_type_name(), "Failed to get interace")
20
21         if (!uvm_config_db #(common_cfg)::get(this, "", "m_cfg", m_cfg))
22             `uvm_fatal(get_type_name(), "Failed to get common_cfg")
23
24         `uvm_info(get_type_name(), "Inside Driver [Build]", UVM_LOW)
25     endfunction
26
27     task run_phase(uvm_phase phase);
28         forever begin
29             seq_item_port.get_next_item(req);
30             drive(req);
31             seq_item_port.item_done();
32         end
33     endtask
34
35     task initialize();
36
37         axi_if.AWADDR = '0;
38         axi_if.AWLEN = '0;
39         axi_if.AWSIZE = '0;
40         axi_if.AWVALID = 1'b0;
41         axi_if.WDATA = '0;
42         axi_if.WLAST = 1'b0;
43         axi_if.WVALID = 1'b0;
44         axi_if.BREADY = 1'b0;
45         axi_if.ARADDR = '0;
46         axi_if.ARLLEN = '0;
47         axi_if.ARSIZE = '0;
48         axi_if.ARVALID = 1'b0;
49         axi_if.RREADY = 1'b0;
50         while (axi_if.ARESETn != 1'b1)
51             @(posedge axi_if.ACLK);
52     endtask

```

AXI DRIVER - PART 1 Driver setup

```

task write(axi4_transaction sig);
    @(posedge axi_if.ACLK);
    @(posedge axi_if.ACLK);
    //Write address
    axi_if.AWADDR <= sig.AWADDR;
    axi_if.AWLEN <= sig.AWLEN;
    axi_if.AWSIZE <= sig.AWSIZE;
    axi_if.AWVALID <= 1'b1;
    do begin
        @(posedge axi_if.ACLK);
    end
    while (!(
        (axi_if.AWREADY === 1'b1) &&
        (axi_if.AWVALID === 1'b1)
    ));
    axi_if.AWVALID <= 1'b0;

    // Write data
    for (int i = 0 ; i < sig.AWLEN + 1 ; i++) begin
        axi_if.WDATA <= sig.WDATA[i];
        axi_if.WVALID <= 1'b1;

        if (i == sig.AWLEN)
            axi_if.WLAST <= 1'b1;
        else
            axi_if.WLAST <= 1'b0;

        do begin
            @(posedge axi_if.ACLK);
        end
        while (!(
            (axi_if.WREADY === 1'b1) &&
            (axi_if.WVALID === 1'b1)
        ));
    end
    axi_if.WVALID <= 1'b0;
    axi_if.WLAST <= 1'b0;

    // Write response
    axi_if.BREADY <= 1'b1;
    do begin
        @(posedge axi_if.ACLK);
    end
    while (!(
        (axi_if.BVALID === 1'b1) &&
        (axi_if.BREADY === 1'b1)
    ));
    axi_if.BREADY <= 1'b0;
    @(posedge axi_if.ACLK);
    @(posedge axi_if.ACLK);

```

AXI DRIVER - PART 2 Write path

```

108 task read(axi4_transaction sig);
109
110 // Read address
111 axi_if.ARADDR <= sig.ARADDR;
112 axi_if.ARLLEN <= sig.ARLLEN;
113 axi_if.ARSIZE <= sig.ARSIZE;
114 axi_if.ARVALID <= 1'b1;
115 do begin
116     @(posedge axi_if.ACLK);
117 end
118 while (!(
119     (axi_if.ARREADY === 1'b1) &&
120     (axi_if.ARVALID === 1'b1)
121 ));
122 axi_if.ARVALID <= 1'b0;
123
124 // Read data
125 axi_if.RREADY <= 1'b1;
126 for (int i = 0 ; i < sig.ARLLEN + 1; i++) begin
127     do begin
128         @(posedge axi_if.ACLK);
129     end
130     while (!(
131         (axi_if.RVALID === 1'b1) &&
132         (axi_if.RREADY === 1'b1)
133     ));
134     `uvm_info(get_type_name,$sformatf(
135         "[DRV][READ] beat=%0d data=%p RLAST=%b",
136         i,
137         axi_if.RDATA,
138         axi_if.RLAST), UVM_LOW);
139     end
140     axi_if.RREADY <= 1'b0;
141 endtask
142
143
144 task drive(axi4_transaction sig);
145     initialize();
146     write(sig);
147     read(sig);
148 endtask
149 endclass
150 endpackage

```

AXI DRIVER - PART 3 Read path


```

1  package mem_monitor_pkg;
2  `include "uvm_macros.svh"
3  import uvm_pkg::*;
4  import mem_transaction_pkg::*;
5
6  class mem_monitor extends uvm_monitor;
7
8      `uvm_component_utils(mem_monitor)
9
10     virtual mem_if mem_vif;
11     uvm_analysis_port #(mem_transaction) a_port;
12
13     function new(string name = "mem_monitor", uvm_component parent);
14         super.new(name, parent);
15     endfunction
16
17     function void build_phase(uvm_phase phase);
18         super.build_phase(phase);
19
20         if (!uvm_config_db #(virtual mem_if)::get(this, "", "vif", mem_vif))
21             `uvm_fatal(get_type_name(), "Failed to get memory interface")
22
23         a_port = new("a_port", this);
24     endfunction
25
26     task run_phase(uvm_phase phase);
27         mem_transaction tr;
28         forever begin
29             @(posedge mem_vif.clk);
30             if (mem_vif.rst_n && mem_vif.mem_en) begin
31                 tr = mem_transaction::type_id::create("tr");
32                 tr.rst_n      = mem_vif.rst_n;
33                 tr.mem_en     = mem_vif.mem_en;
34                 tr.mem_we     = mem_vif.mem_we;
35                 tr.mem_addr   = mem_vif.mem_addr;
36                 tr.mem_wdata  = mem_vif.mem_wdata;
37                 if (!mem_vif.mem_we) begin
38                     @(posedge mem_vif.clk);
39                     tr.mem_rdata = mem_vif.mem_rdata;
40                 end
41                 a_port.write(tr);
42             end
43         end
44     endtask
45 endclass
46
47 endpackage

```

MEMORY MONITOR Memory monitor

```

1  package axi4_monitor_pkg;
2  `include "uvm_macros.svh"
3  import uvm_pkg::*;
4  import common_cfg_pkg::*;
5  import axi4_transaction_pkg::*;
6
7
8
9  class axi4_monitor extends uvm_monitor;
10
11     `uvm_component_utils(axi4_monitor)
12
13     virtual axi4_if axi_if;
14     common_cfg m_cfg;
15     uvm_analysis_port #(axi4_transaction) a_port;
16
17
18     function new (string name = "axi4_monitor", uvm_component parent);
19         super.new(name, parent);
20     endfunction
21
22     function void build_phase(uvm_phase phase);
23         super.build_phase(phase);
24
25         if (!uvm_config_db #(virtual axi4_if)::get(this, "", "vif", axi_if))
26             `uvm_fatal(get_type_name(), "Failed to get interface")
27
28         if (!uvm_config_db #(common_cfg)::get(this, "", "m_cfg", m_cfg))
29             `uvm_fatal(get_type_name(), "Failed to get common_cfg")
30
31         a_port = new("a_port", this);
32
33         `uvm_info(get_type_name(), "Inside Monitor [Build]", UVM_LOW)
34     endfunction
35
36     task run_phase(uvm_phase phase);
37         `uvm_info(get_type_name(), "Inside Monitor [Run]", UVM_LOW)
38         forever begin
39             monitor();
40         end
41     endtask

```

AXI MONITOR - PART 1 AXI monitor

```

43 task monitor_write();
44 axi4_transaction tr;
45 while (axi_if.ARESETn != 1'b1)
46   @(posedge axi_if.ACLK);
47   forever begin
48     tr = new();
49     tr.operation = WRITE_OP;
50     do begin
51       @(posedge axi_if.ACLK);
52     end while (!(
53       (axi_if.ARESETn == 1'b1) &&
54       (axi_if.AWVALID == 1'b1) &&
55       (axi_if.AWREADY == 1'b1)
56     ));
57     tr.AWADDR = axi_if.AWADDR;
58     tr.AWLEN = axi_if.AWLEN;
59     tr.AWSIZE = axi_if.AWSIZE;
60     tr.WDATA = new[tr.AWLEN + 1];
61     for (int i = 0 ; i < tr.AWLEN + 1; i++) begin
62       do begin
63         @(posedge axi_if.ACLK);
64       end while (!(
65         (axi_if.ARESETn == 1'b1) &&
66         (axi_if.WVALID == 1'b1) &&
67         (axi_if.WREADY == 1'b1)
68       ));
69       tr.WDATA[i] = axi_if.WDATA;
70       if (i == tr.AWLEN)
71         tr.WLAST = axi_if.WLAST;
72     end
73     tr.WLAST = axi_if.WLAST;
74     do begin
75       @(posedge axi_if.ACLK);
76     end while (!(
77       (axi_if.ARESETn == 1'b1) &&
78       (axi_if.BVALID == 1'b1) &&
79       (axi_if.BREADY == 1'b1)
80     ));
81     tr.BRESP = axi_if.BRESP;
82     `uvm_info(get_type_name(), $sformatf("[MON][WRITE] addr=%h data=%p last=%b resp=%b \n",
83       tr.AWADDR, tr.WDATA, tr.WLAST, tr.BRESP), UVM_LOW);
84     a_port.write(tr);
85   end
86 endtask

```

AXI MONITOR - PART 2 AXI monitor

```

134 task monitor();
135   fork
136     monitor_write();
137     monitor_read();
138   join
139   endtask
140 endclass
141 endpackage
142

```

AXI MONITOR - PART 4 Run task

```

88     task monitor_read();
89         axi4_transaction tr;
90
91         while (axi_if.ARESETn !== 1'b1)
92             @(posedge axi_if.ACLK);
93
94         forever begin
95             tr = new();
96             tr.operation = READ_OP;
97             do begin
98                 @(posedge axi_if.ACLK);
99             end while (!(
100                 (axi_if.ARESETn === 1'b1) &&
101                 (axi_if.ARVALID === 1'b1) &&
102                 (axi_if.ARREADY === 1'b1)
103             ));
104
105             tr.ARADDR = axi_if.ARADDR;
106             tr.ARLLEN = axi_if.ARLLEN;
107             tr.ARSIZE = axi_if.ARSIZE;
108
109             tr.RDATA = new[tr.ARLLEN + 1];
110             for (int i = 0 ; i < tr.ARLLEN + 1; i++) begin
111                 do begin
112                     @(posedge axi_if.ACLK);
113                 end while (!(
114                     (axi_if.ARESETn === 1'b1) &&
115                     (axi_if.RVALID === 1'b1) &&
116                     (axi_if.RREADY === 1'b1)
117                 ));
118                 tr.RDATA[i] = axi_if.RDATA;
119                 tr.RRESP = axi_if.RRESP;
120
121                 if (i == tr.ARLLEN)
122                     tr.RLAST = axi_if.RLAST;
123             end
124             `uvm_info(get_type_name(), $sformatf("[MON][READ] addr=%h len=%0d data=%p last=%b resp=%b",
125                 tr.ARADDR,
126                 tr.ARLLEN,
127                 tr.RDATA,
128                 tr.RLAST,
129                 tr.RRESP), UVM_LOW);
130             a_port.write(tr);
131         end
132     endtask

```

AXI MONITOR - PART 3 AXI monitor

```

1 package mem_agent_pkg;
2 `include "uvm_macros.svh"
3 import uvm_pkg::*;
4 import mem_monitor_pkg::*;
5
6 class mem_agent extends uvm_agent;
7
8     `uvm_component_utils(mem_agent)
9
10
11     mem_monitor mon;
12
13     function new (string name = "mem_agent", uvm_component parent);
14         super.new(name, parent);
15     endfunction
16
17     function void build_phase (uvm_phase phase);
18         super.build_phase(phase);
19         `uvm_info(get_type_name(), "Inside Agent [Build]", UVM_LOW)
20         mon = mem_monitor::type_id::create("mon", this);
21     endfunction
22
23 endclass
24 endpackage
25

```

MEMORY AGENT Memory agent

```

1 package axi4_agent_pkg;
2 `include "uvm_macros.svh"
3 import uvm_pkg::*;
4 import axi4_sequencer_pkg::*;
5 import axi4_driver_pkg::*;
6 import axi4_monitor_pkg::*;
7
8 class axi4_agent extends uvm_agent;
9
10     `uvm_component_utils(axi4_agent)
11     uvm_active_passive_enum is_active = UVM_ACTIVE;
12
13     axi4_sequencer sqr;
14     axi4_driver drv;
15     axi4_monitor mon;
16
17     function new (string name = "axi4_agent", uvm_component parent);
18         super.new(name, parent);
19     endfunction
20
21     function void build_phase (uvm_phase phase);
22         super.build_phase(phase);
23         `uvm_info(get_type_name(), "Inside Agent [Build]", UVM_LOW)
24
25         if (!uvm_config_db #(uvm_active_passive_enum)::get(this, "", "is_active", is_active))
26             `uvm_fatal(get_type_name(), "Failed to get enum")
27
28         mon = axi4_monitor::type_id::create("mon", this);
29
30         if (is_active == UVM_ACTIVE) begin
31             drv = axi4_driver::type_id::create("drv", this);
32             sqr = axi4_sequencer::type_id::create("sqr", this);
33         end
34     endfunction
35
36     function void connect_phase (uvm_phase phase);
37         super.connect_phase(phase);
38         if (is_active == UVM_ACTIVE) begin
39             drv.seq_item_port.connect(sqr.seq_item_export);
40         end
41     endfunction
42
43 endclass
44 endpackage
45

```

AXI AGENT AXI agent

```

50         writes_checked++;
51         reference_memory[tr.mem_addr] = tr.mem_wdata;
52         passed++;
53         `uvm_info(get_type_name(),$formatf("[MEM_SCB][WRITE][PASS] addr=%0h data=%0h", tr.mem_addr,tr.mem_wdata),UVM_LOW)
54     end
55     else if (tr.mem_we == 1'b0) begin
56         reads_checked++;
57         if (tr.mem_rdata == reference_memory[tr.mem_addr]) begin
58             passed++;
59             `uvm_info(get_type_name(), $formatf("[MEM_SCB][READ][PASS] addr=%0h expected=%0h actual=%0h",
60             ,tr.mem_addr
61             ,reference_memory[tr.mem_addr],
62             tr.mem_rdata),UVM_LOW)
63         end
64     else begin
65         failed++;
66         `uvm_error(get_type_name(),$formatf("[MEM_SCB][READ][FAIL] addr=%0h expected=%0h actual=%0h",
67         ,tr.mem_addr
68         ,reference_memory[tr.mem_addr]
69         ,tr.mem_rdata))
70     end
71 end
72 end
73 endtask
74
75 function void extract_phase(uvm_phase phase);
76     super.extract_phase(phase);
77
78     `uvm_info(get_type_name(), "=====",UVM_LOW)
79     `uvm_info(get_type_name(),"MEMORY SCOREBOARD REPORT",UVM_LOW)
80     `uvm_info(get_type_name(),$formatf("Writes checked      : %0d",writes_checked), UVM_LOW)
81     `uvm_info(get_type_name(), $formatf("Reads checked       : %0d", reads_checked),UVM_LOW)
82     `uvm_info(get_type_name(),$formatf("Passed transactions: %0d", passed), UVM_LOW)
83     `uvm_info(get_type_name(),$formatf("Failed transactions: %0d", failed),UVM_LOW)
84     `uvm_info(get_type_name(),$formatf("Total transactions : %0d",total_transactions),UVM_LOW)
85     `uvm_info(get_type_name(),"=====", UVM_LOW)
86
87 endfunction
88 endclass
89 endpackage

```

MEMORY SCOREBOARD - PART 1 Memory checking

```

1 package mem_scoreboard_pkg;
2
3 class mem_scoreboard extends uvm_scoreboard;
4     `uvm_component_utils(mem_scoreboard)
5     uvm_analysis_export #(mem_transaction) analysis_exp;
6     uvm_tlm_analysis_fifo #(mem_transaction) fifo;
7     logic [31:0] reference_memory [0:1023];
8     int unsigned writes_checked;
9     int unsigned reads_checked;
10    int unsigned passed;
11    int unsigned failed;
12    int unsigned total_transactions;
13
14    function new(string name, uvm_component parent);
15        super.new(name, parent);
16        writes_checked = 0;
17        reads_checked = 0;
18        passed = 0;
19        failed = 0;
20        total_transactions = 0;
21        for (int i = 0; i < 1024; i++)
22            reference_memory[i] = 32'd0;
23        analysis_exp = new("analysis_exp", this);
24        fifo = new("fifo", this);
25    endfunction
26
27    function void build_phase(uvm_phase phase);
28        super.build_phase(phase);
29        `uvm_info(get_type_name(),"Inside Memory Scoreboard [Build]",UVM_LOW)
30    endfunction
31
32    function void connect_phase(uvm_phase phase);
33        super.connect_phase(phase);
34        analysis_exp.connect(fifo.analysis_export);
35    endfunction
36
37    task run_phase(uvm_phase phase);
38        mem_transaction tr;
39        `uvm_info(get_type_name(),"Inside Memory Scoreboard [Run]",UVM_LOW)
40        forever begin
41            fifo.get(tr);
42            total_transactions++;
43            if (tr.mem_we == 1'b1) begin
44                writes_checked++;
45                reference_memory[tr.mem_addr] = tr.mem_wdata;
46                passed++;
47                `uvm_info(get_type_name(),$formatf("[MEM_SCB][WRITE][PASS] addr=%0h data=%0h", tr.mem_addr,tr.mem_wdata),UVM_LOW)
48            end

```

MEMORY SCOREBOARD - PART 2 Memory checking

```

1  package axi4_scoreboard_pkg;
2
3  `include "uvm_macros.svh"
4
5  logic [31:0] reference_memory [0:1023];
6
7  class axi4_scoreboard extends uvm_scoreboard;
8
9
10     `uvm_component_utils(axi4_scoreboard)
11
12     uvm_analysis_export #(axi4_transaction) analysis_exp;
13     uvm_tlm_analysis_fifo #(axi4_transaction) fifo;
14
15     int unsigned writes_checked;
16     int unsigned reads_checked;
17     int unsigned passed;
18     int unsigned failed;
19     int unsigned total_transactions;
20
21
22     function new (string name , uvm_component parent);
23         super.new(name,parent);
24
25         for (int i = 0; i < 1024; i++)
26             reference_memory[i] = 32'd0;
27
28         writes_checked = 0;
29         reads_checked = 0;
30         passed = 0;
31         failed = 0;
32         total_transactions = 0;
33         analysis_exp = new("analysis_exp", this);
34         fifo = new("fifo", this);
35     endfunction
36
37     function void build_phase (uvm_phase phase);
38         super.build_phase(phase);
39         `uvm_info(get_type_name(), "Inside Scoreboard", UVM_LOW)
40     endfunction
41
42     function void connect_phase(uvm_phase phase);
43         analysis_exp.connect(fifo.analysis_export);
44     endfunction
45
46
47     task run_phase(uvm_phase phase);
48         `uvm_info(get_type_name(), "Inside Monitor [Run]", UVM_LOW)
49         scoreboard();
50     endtask
51
52     function void extract_phase(uvm_phase phase);
53         `uvm_info(get_type_name(), "=====", UVM_LOW);
54         `uvm_info(get_type_name(), "AXI SCOREBOARD REPORT", UVM_LOW);
55         `uvm_info(get_type_name(), $sformatf("Writes checked      : %0d", writes_checked), UVM_LOW);
56         `uvm_info(get_type_name(), $sformatf("Reads checked       : %0d", reads_checked), UVM_LOW);
57         `uvm_info(get_type_name(), $sformatf("Passed transactions: %0d", passed), UVM_LOW);
58         `uvm_info(get_type_name(), $sformatf("Failed transactions: %0d", failed), UVM_LOW);
59         `uvm_info(get_type_name(), "=====", UVM_LOW);
60     endfunction

```

AXI SCOREBOARD - PART 1 AXI checking

```

62     function bit legal_write(axi4_transaction tr);
63
64         int unsigned beats;
65         int unsigned bytes_per_beat;
66         int unsigned last_byte_addr;
67
68         beats = (tr.AWLEN) + 1;
69         bytes_per_beat = 1 << tr.AWSIZE;
70
71         last_byte_addr =
72             int'(tr.AWADDR) +
73             (beats * bytes_per_beat) - 1;
74
75         return (tr.AWSIZE == 3'd2) && (tr.AWADDR[1:0] == 2'b00) && (tr.AWADDR <= 16'h0FFC) &&
76             (last_byte_addr <= 16'h0FFF) && ((last_byte_addr >> 2) < 1024);
77
78     endfunction
79
80     function bit legal_read(axi4_transaction tr);
81
82         int unsigned beats;
83         int unsigned bytes_per_beat;
84         int unsigned last_byte_addr;
85
86         beats = (tr.ARLEN) + 1;
87         bytes_per_beat = 1 << tr.ARSIZE;
88
89         last_byte_addr =
90             int'(tr.ARADDR) +
91             (beats * bytes_per_beat) - 1;
92
93         return (tr.ARSIZE == 3'd2) && (tr.ARADDR[1:0] == 2'b00) && (tr.ARADDR <= 16'h0FFC) &&
94             (last_byte_addr <= 16'h0FFF) && ((last_byte_addr >> 2) < 1024);
95
96     endfunction
97

```

AXI SCOREBOARD - PART 2 AXI checking

```

task check_write(axi4_transaction tr);

    bit transaction_pass;
    bit legal;
    bit data_count_ok;
    logic [1:0] expected_response;
    int unsigned index;
    int unsigned beats;
    writes_checked++;
    transaction_pass = 1'b1;
    beats = (tr.AWLEN) + 1;
    legal = legal_write(tr);
    expected_response =
        legal ? 2'b00 : 2'b10;
    index = tr.AWADDR >> 2;

    if (tr.BRESP != expected_response) begin
        `uvm_error(get_type_name(), $sprintf("[SCB][WRITE] Response mismatch: addr=%h expected=%b actual=%b", tr.AWADDR, expected_response, tr.BRESP));
        transaction_pass = 1'b0;
    end

    data_count_ok = (tr.WDATA.size() == beats);

    if (!data_count_ok) begin
        `uvm_error(get_type_name(), $sprintf("[SCB][WRITE] Beat count mismatch: expected=%0d actual=%0d", beats, tr.WDATA.size()));
        transaction_pass = 1'b0;
    end

    if (legal && (tr.WLAST != 1'b1)) begin
        `uvm_error(get_type_name(), $sprintf("[SCB][WRITE] WLAST not asserted on final beat: addr=%h len=%0d", tr.AWADDR, tr.AWLEN));
        transaction_pass = 1'b0;
    end

    if (legal && (tr.BRESP == 2'b00) && data_count_ok)
    begin
        for (int i = 0; i < beats; i++) begin
            reference_memory[index + i] =
                tr.WDATA[i];
        end
    end

    if (transaction_pass) begin
        passed++;
        `uvm_info(get_type_name(), $sprintf("[SCB][WRITE][PASS] addr=%h len=%0d index=%0d data=%p resp=%b", tr.AWADDR,
            tr.AWLEN,
            index,
            tr.WDATA,
            tr.BRESP), UVM_LOW);
    end
    else begin
        failed++;
    end
endtask

```

AXI SCOREBOARD - PART 3 AXI checking

```

task check_read(axi4_transaction tr);

    bit transaction_pass;
    bit legal;
    bit data_count_ok;
    logic [1:0] expected_response;
    logic [31:0] expected_data;
    int unsigned index;
    int unsigned beats;
    int unsigned beat_addr;

    reads_checked++;
    transaction_pass = 1'b1;
    beats = int'(tr.ARLEN) + 1;
    legal = legal_read(tr);
    expected_response = legal ? 2'b00 : 2'b10;
    index = tr.ARADDR >> 2;

    if (tr.RRESP != expected_response) begin
        `uvm_error(get_type_name(), $sprintf("[SCB][READ] Response mismatch: addr=%h expected=%b actual=%b",
            tr.ARADDR,
            expected_response,
            tr.RRESP));
        transaction_pass = 1'b0;
    end

    if (legal) begin
        data_count_ok = (tr.RDATA.size() == beats);
        if (!data_count_ok) begin
            `uvm_error(get_type_name(), $sprintf("[SCB][READ] Beat count mismatch: expected=%0d actual=%0d",
                beats,
                tr.RDATA.size()));
            transaction_pass = 1'b0;
        end
        else begin
            for (int i = 0; i < beats; i++) begin
                expected_data = reference_memory[index + i];
                beat_addr = (tr.ARADDR) + (i << tr.ARSIZE);
                if (tr.RDATA[i] != expected_data) begin
                    `uvm_error(get_type_name(), $sprintf("[SCB][READ] Data mismatch: beat=%0d addr=%h expected=%p actual=%p",
                        i,
                        beat_addr,
                        expected_data,
                        tr.RDATA[i]));
                    transaction_pass = 1'b0;
                end
            end
        end
    end
    else begin
        for (int i = 0; i < tr.RDATA.size(); i++) begin
            if (tr.RDATA[i] != 32'd0) begin
                `uvm_error(get_type_name(), $sprintf("[SCB][READ] Invalid read must return zero data: beat=%0d actual=%h",
                    i,
                    tr.RDATA[i]));
                transaction_pass = 1'b0;
            end
        end
    end
endtask

```

AXI SCOREBOARD - PART 4 AXI checking

```

        end
    end

    if (tr.RLAST !== 1'b1) begin
        `uvm_error(get_type_name(), $sformatf("[SCB][READ] RLAST not asserted on final returned beat: addr=%h len=%0d",
            tr.ARADDR,
            tr.ARLEN));
        transaction_pass = 1'b0;
    end

    if (transaction_pass) begin
        passed++;
        `uvm_info(get_type_name(), $sformatf("[SCB][READ][PASS] addr=%h len=%0d index=%0d data=%p resp=%b",
            tr.ARADDR,
            tr.ARLEN,
            index,
            tr.RDATA,
            tr.RRESP), UVM_LOW);
    end
    else begin
        failed++;
    end
endtask

task scoreboard();
    axi4_transaction actual;
    forever begin
        fifo.get(actual);
        case (actual.operation)
            WRITE_OP: check_write(actual);
            READ_OP : check_read(actual);
            default : begin
                failed++;
                `uvm_error(get_type_name(), "[SCB] Unknown transaction operation");
            end
        endcase

        total_transactions++;
    end
endtask
endclass
endpackage

```



```

58     c_RRESP : coverpoint RRESP {
59         bins okay    = {2'b00};
60         ignore_bins slverr = {2'b10};
61     }
62
63     c_AWSIZE : coverpoint AWSIZE {
64         bins word = {3'd2};
65     }
66
67     c_ARSIZE : coverpoint ARSIZE {
68         bins word = {3'd2};
69     }
70
71 endgroup
72
73 uvm_analysis_export #(axi4_transaction) analysis_exp;
74 uvm_tlm_analysis_fifo #(axi4_transaction) fifo;
75
76 function new(string name, uvm_component parent);
77     super.new(name,parent);
78     analysis_exp = new("analysis_exp", this);
79     fifo = new("fifo", this);
80     cg = new();
81 endfunction
82
83 function void connect_phase(uvm_phase phase);
84     super.connect_phase(phase);
85     analysis_exp.connect(fifo.analysis_export);
86 endfunction
87
88 task run_phase(uvm_phase phase);
89     axi4_transaction tr;
90     forever begin
91         fifo.get(tr);
92         AWADDR = tr.AWADDR;
93         ARADDR = tr.ARADDR;
94         AWLEN = tr.AWLEN;
95         ARLEN = tr.ARLEN;
96         BRESP = tr.BRESP;
97         RRESP = tr.RRESP;
98         AWSIZE = tr.AWSIZE;
99         ARSIZE = tr.ARSIZE;
100         cg.sample();
101     end
102 endtask

```

AXI COVERAGE - PART 1 Coverage model

```

6 class axi4_cov extends uvm_component;
12 logic [1:0] BRESP;
13 logic [1:0] RRESP;
14 logic [2:0] AWSIZE;
15 logic [2:0] ARSIZE;
16
17 covergroup cg;
18     c_AWADDR : coverpoint AWADDR {
19         bins b1 = {[16'd0 : 16'd511]};
20         bins b2 = {[16'd512 : 16'd1023]};
21         bins b3 = {[16'd1024 : 16'd1535]};
22         bins b4 = {[16'd1536 : 16'd2047]};
23         bins b5 = {[16'd2048 : 16'd2559]};
24         bins b6 = {[16'd2560 : 16'd3071]};
25         bins b7 = {[16'd3072 : 16'd3583]};
26         bins b8 = {[16'd3584 : 16'd4092]};
27     }
28     c_ARADDR : coverpoint ARADDR {
29         bins b1 = {[16'd0 : 16'd511]};
30         bins b2 = {[16'd512 : 16'd1023]};
31         bins b3 = {[16'd1024 : 16'd1535]};
32         bins b4 = {[16'd1536 : 16'd2047]};
33         bins b5 = {[16'd2048 : 16'd2559]};
34         bins b6 = {[16'd2560 : 16'd3071]};
35         bins b7 = {[16'd3072 : 16'd3583]};
36         bins b8 = {[16'd3584 : 16'd4092]};
37     }
38     c_AWLEN : coverpoint AWLEN {
39         bins b0 = {8'd0};
40         bins b1 = {[8'd1 : 8'd63]};
41         bins b2 = {[8'd64 : 8'd127]};
42         bins b3 = {[8'd128 : 8'd191]};
43         bins b4 = {[8'd192 : 8'd255]};
44     }
45     c_ARLEN : coverpoint ARLEN {
46         bins b0 = {8'd0};
47         bins b1 = {[8'd1 : 8'd63]};
48         bins b2 = {[8'd64 : 8'd127]};
49         bins b3 = {[8'd128 : 8'd191]};
50         bins b4 = {[8'd192 : 8'd255]};
51     }
52
53     c_BRESP : coverpoint BRESP {
54         bins okay    = {2'b00};
55         ignore_bins slverr = {2'b10};
56     }

```

AXI COVERAGE - PART 2 Covergroups and sampling

```

1  package axi4_env_pkg;
2  import uvm_pkg::*;
3  import axi4_agent_pkg::*;
4  import axi4_scoreboard_pkg::*;
5  import axi4_coverage_pkg::*;
6  import common_cfg_pkg::*;
7  import mem_agent_pkg::*;
8  import mem_scoreboard_pkg::*;
9  `include "uvm_macros.svh"
10
11  class axi4_env extends uvm_env;
12      `uvm_component_utils(axi4_env)
13      axi4_agent    agt;
14      axi4_scoreboard scb;
15      axi4_cov      cov;
16      mem_agent     mem_agt;
17      mem_scoreboard mem_scb;
18      common_cfg m_cfg;
19
20      function new (string name , uvm_component parent);
21          super.new(name,parent);
22      endfunction
23
24      function void build_phase (uvm_phase phase);
25          super.build_phase(phase);
26
27          agt = axi4_agent::type_id::create("agt",this);
28          scb = axi4_scoreboard::type_id::create("scb",this);
29          cov = axi4_cov::type_id::create("cov",this);
30          mem_agt = mem_agent::type_id::create("mem_agt",this);
31          mem_scb = mem_scoreboard::type_id::create("mem_scb",this);
32
33          `uvm_info(get_type_name(), "Inside Env [Build]", UVM_LOW)
34
35          uvm_config_db #(common_cfg)::get(this,"", "m_cfg",m_cfg);
36
37      endfunction
38
39      function void connect_phase (uvm_phase phase);
40          super.connect_phase(phase);
41          agt.mon.a_port.connect(scb.analysis_exp);
42          agt.mon.a_port.connect(cov.analysis_exp);
43          mem_agt.mon.a_port.connect(mem_scb.analysis_exp);
44          `uvm_info(get_type_name(), "Inside Env [Connect]", UVM_LOW)
45      endfunction
46  endclass
47  endpackage

```

```

1  package axi4_test_pkg;
2
3  import uvm_pkg::*;
4  `include "uvm_macros.svh"
5
6  import axi4_env_pkg::*;
7  import axi4_sequence_pkg::*;
8  import common_cfg_pkg::*;
9
10 class axi4_test extends uvm_test;
11
12     axi4_env env;
13     axi4_sequence seq;
14     common_cfg m_cfg;
15
16     `uvm_component_utils(axi4_test)
17
18     function new (string name , uvm_component parent);
19         super.new(name,parent);
20     endfunction
21
22     function void build_phase (uvm_phase phase);
23         m_cfg = new();
24         uvm_config_db #(common_cfg)::set(this,"env.agt.*",m_cfg,m_cfg);
25
26         env = axi4_env::type_id::create("env", this);
27         seq = axi4_sequence::type_id::create("seq", this);
28
29         super.build_phase(phase);
30         `uvm_info(get_type_name(),"Inside test phase",UVM_LOW)
31     endfunction
32
33     task run_phase(uvm_phase phase);
34
35         phase.raise_objection(this);
36         `uvm_info(get_type_name(), "Sequence begin", UVM_LOW)
37         seq.start(env.agt.sqr);
38         `uvm_info(get_type_name(), "Sequence finished", UVM_LOW)
39         phase.drop_objection(this);
40
41     endtask
42 endclass
43 endpackage
44

```

AXI TEST UVM test

```

1  import axi4_pkg::*;
2  import mem_transaction_pkg::*;
3  import uvm_pkg::*;
4  `include "uvm_macros.svh"
5
6  module top;
7      axi4_if axi4_vif();
8      mem_if  mem_vif();
9
10     axi4 dut (
11         .ACLK(axi4_vif.ACLK), .ARESETn(axi4_vif.ARESETn),
12         .AWADDR(axi4_vif.AWADDR), .AWLEN(axi4_vif.AWLEN),
13         .AWSIZE(axi4_vif.AWSIZE), .AWVALID(axi4_vif.AWVALID),
14         .AWREADY(axi4_vif.AWREADY), .WDATA(axi4_vif.WDATA),
15         .WLAST(axi4_vif.WLAST), .WVALID(axi4_vif.WVALID),
16         .WREADY(axi4_vif.WREADY), .BRESP(axi4_vif.BRESP),
17         .BVALID(axi4_vif.BVALID), .BREADY(axi4_vif.BREADY),
18         .ARADDR(axi4_vif.ARADDR), .ARLEN(axi4_vif.ARLEN),
19         .ARSIZE(axi4_vif.ARSIZE), .ARVALID(axi4_vif.ARVALID),
20         .ARREADY(axi4_vif.ARREADY), .RDATA(axi4_vif.RDATA),
21         .RRESP(axi4_vif.RRESP), .RLAST(axi4_vif.RLAST),
22         .RVALID(axi4_vif.RVALID), .RREADY(axi4_vif.RREADY)
23     );
24
25     assign mem_vif.clk      = axi4_vif.ACLK;
26     assign mem_vif.rst_n   = axi4_vif.ARESETn;
27     assign mem_vif.mem_en  = dut.mem_en;
28     assign mem_vif.mem_we  = dut.mem_we;
29     assign mem_vif.mem_addr = dut.mem_addr;
30     assign mem_vif.mem_wdata = dut.mem_wdata;
31     assign mem_vif.mem_rdata = dut.mem_rdata;
32
33     always #5 axi4_vif.ACLK = ~axi4_vif.ACLK;
34
35     initial begin
36         axi4_vif.ACLK = 0;
37         axi4_vif.ARESETn = 0;
38         #5
39         axi4_vif.ARESETn = 1;
40     end
41
42     initial begin
43         uvm_config_db #(uvm_active_passive_enum)::set(null, "uvm_test_top.env.agt", "is_active", UVM_ACTIVE);
44         uvm_config_db #(virtual axi4_if)::set(null, "uvm_test_top.env.agt.*", "vif", axi4_vif);
45         uvm_config_db #(virtual mem_if)::set(null, "uvm_test_top.env.mem.agt.*", "vif", mem_vif);
46         run_test("axi4_test");
47     end
48 endmodule

```

TOP MODULE Top-level integration

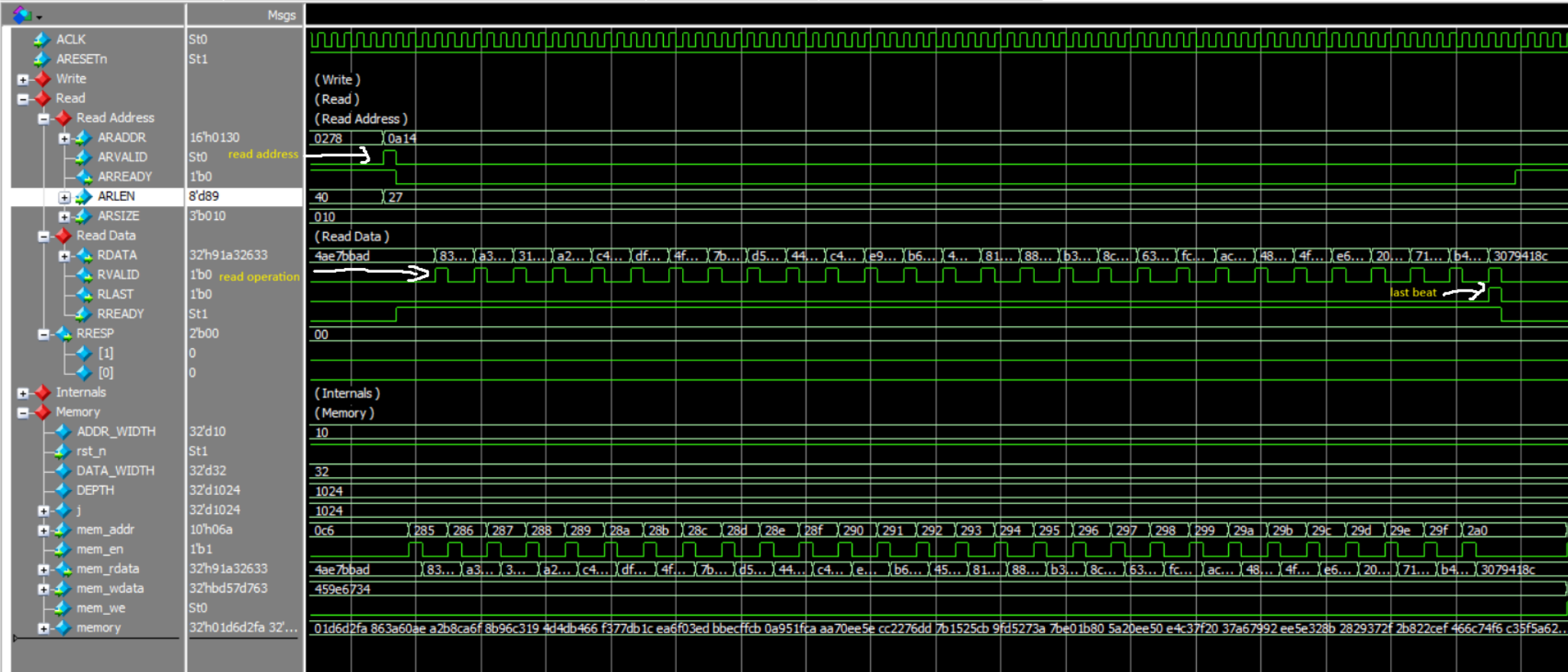
```
# UVM_INFO axi4_test.sv(38) @ 9888665: uvm_test_top [axi4_test] Sequence finished
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1267) @ 9888665: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
# UVM_INFO mem_scoreboard.sv(80) @ 9888665: uvm_test_top.env.mem_scb [mem_scoreboard] =====
# UVM_INFO mem_scoreboard.sv(81) @ 9888665: uvm_test_top.env.mem_scb [mem_scoreboard] MEMORY SCOREBOARD REPORT
# UVM_INFO mem_scoreboard.sv(82) @ 9888665: uvm_test_top.env.mem_scb [mem_scoreboard] Writes checked : 244509
# UVM_INFO mem_scoreboard.sv(83) @ 9888665: uvm_test_top.env.mem_scb [mem_scoreboard] Reads checked : 242119
# UVM_INFO mem_scoreboard.sv(84) @ 9888665: uvm_test_top.env.mem_scb [mem_scoreboard] Passed transactions: 486628
# UVM_INFO mem_scoreboard.sv(85) @ 9888665: uvm_test_top.env.mem_scb [mem_scoreboard] Failed transactions: 0
# UVM_INFO mem_scoreboard.sv(86) @ 9888665: uvm_test_top.env.mem_scb [mem_scoreboard] Total transactions : 486628
# UVM_INFO mem_scoreboard.sv(87) @ 9888665: uvm_test_top.env.mem_scb [mem_scoreboard] =====
# UVM_INFO axi4_scoreboard.sv(53) @ 9888665: uvm_test_top.env.scb [axi4_scoreboard] =====
# UVM_INFO axi4_scoreboard.sv(54) @ 9888665: uvm_test_top.env.scb [axi4_scoreboard] AXI SCOREBOARD REPORT
# UVM_INFO axi4_scoreboard.sv(55) @ 9888665: uvm_test_top.env.scb [axi4_scoreboard] Writes checked : 2000
# UVM_INFO axi4_scoreboard.sv(56) @ 9888665: uvm_test_top.env.scb [axi4_scoreboard] Reads checked : 2000
# UVM_INFO axi4_scoreboard.sv(57) @ 9888665: uvm_test_top.env.scb [axi4_scoreboard] Passed transactions: 4000
# UVM_INFO axi4_scoreboard.sv(58) @ 9888665: uvm_test_top.env.scb [axi4_scoreboard] Failed transactions: 0
# UVM_INFO axi4_scoreboard.sv(59) @ 9888665: uvm_test_top.env.scb [axi4_scoreboard] =====
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO : 736781
# UVM_WARNING : 0
# UVM_ERROR : 0
# UVM_FATAL : 0
# ** Report counts by id
# [Questa UVM] 2
# [RNTST] 1
# [TEST_DONE] 1
# [axi4_agent] 1
# [axi4_driver] 242120
# [axi4_env] 2
# [axi4_monitor] 4002
# [axi4_scoreboard] 4009
# [axi4_sequencer] 1
# [axi4_test] 3
# [mem_agent] 1
# [mem_scoreboard] 486638
# ** Note: $finish : E:/questasim64_2021.1/win64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
# Time: 9888665 ns Iteration: 61 Instance: /top
# 1
# Break in Task uvm_pkg/uvm_root::run_test at E:/questasim64_2021.1/win64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh line 430
```

LOG 1 UVM test summary

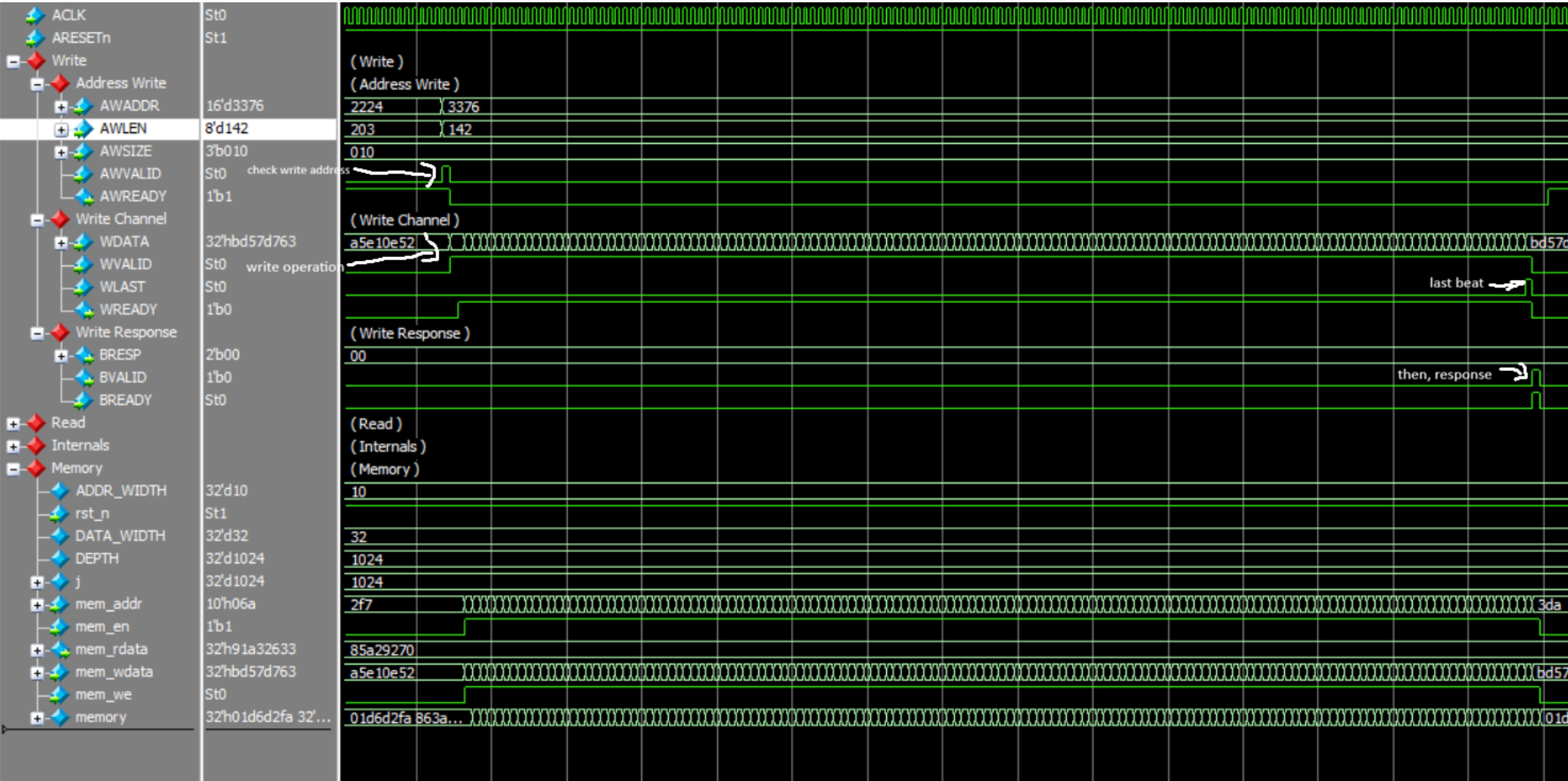
```
# UVM_INFO axi4_monitor.sv(126) @ 9888665: uvm_test_top.env.agt.mon [axi4_monitor] [MON][READ] addr=056c len=180 data='(371380723, 2430818532, 2000493565, 2215289219, 1710889738, 518389197, 3892309253, 1468525934, 3
949844703, 2057717983, 3174374327, 981169109, 3733932212, 380455905, 4002957703, 2988571570, 2912353615, 1838083155, 1190523911, 3428477824, 940123533, 2926923434, 1403717775, 643730012, 2561133240, 2823389094, 2046
563630, 935759128, 3341034706, 1692191823, 1965776832, 1806869450, 2484935683, 3958798018, 362139655, 2427854030, 1153902766, 2978456370, 3252758747, 3405600485, 2496681174, 2659244965, 42154705, 328264661, 27944025
97, 1658709575, 3363521597, 1566336700, 2328802979, 4035958571, 947304315, 3086967152, 2059002863, 3768563607, 3014841494, 3677339158, 1818170437, 2981664208, 3565373238, 3591266993, 4036601283, 1540431038, 10601107
0, 1418552722, 4062997160, 3201346529, 4101823863, 861328203, 2696154175, 580269713, 628273369, 3358883098, 1716637111, 637264964, 1912296794, 3620533734, 98791315, 3945302754, 742672925, 35326136, 1640017426, 37804
40686, 3405744207, 3057665753, 384475274, 1660026994, 2038783194, 801698642, 344371948, 408016060, 2606528644, 172307961, 1325782072, 972711596, 4081456034, 2159691645, 3687743745, 1856805740, 2564135007, 3937135181
, 2503202829, 1632117550, 4036718069, 3988638790, 2578076707, 2508120268, 3182155045, 89324487, 113903438, 3503701767, 32700933, 2910879891, 3590324085, 1743513878, 3260666055, 1053626332, 65308124, 659528255, 15693
37110, 3256796383, 956116232, 3990444024, 304262211, 2159253790, 1115861953, 3933838962, 2547040480, 3047802294, 3938063058, 4224174205, 2159680756, 810755375, 2482400966, 1231106552, 2380411828, 1354282538, 1883243
494, 3580735433, 2427068586, 3785512055, 900480325, 3821731165, 3539884527, 3281565445, 2392742875, 1183068911, 433598458, 3704004061, 1772725673, 2392432239, 1941040794, 2464961769, 370515460, 3336289900, 389464789
5, 1053553301, 1598335332, 3239810836, 1644860006, 1647641675, 3992215759, 4169949752, 2320804757, 1056733854, 2911680706, 3839420050, 2046390357, 1806386831, 3055158653, 309363189, 2957157034, 308404156, 1477980812
, 1604728646, 3756158470, 4278205796, 1148217271, 3859973958, 2746355467, 1652349044, 3130334653) last=1 resp=00
# UVM_INFO axi4_driver.sv(142) @ 9888665: uvm_test_top.env.agt.drv [axi4_driver] [DRV][READ] beat=180 data=3130334653 RLAST=1
# UVM_INFO axi4_scoreboard.sv(231) @ 9888665: uvm_test_top.env.scb [axi4_scoreboard] [SCB][READ][PASS] addr=056c len=180 index=347 data='(371380723, 2430818532, 2000493565, 2215289219, 1710889738, 518389197, 3892309
253, 1468525934, 3949844703, 2057717983, 3174374327, 981169109, 3733932212, 380455905, 4002957703, 2988571570, 2912353615, 1838083155, 1190523911, 3428477824, 940123533, 2926923434, 1403717775, 643730012, 2561133240
, 2823389094, 2046563630, 935759128, 3341034706, 1692191823, 1965776832, 1806869450, 2484935683, 3958798018, 362139655, 2427854030, 1153902766, 2978456370, 3252758747, 3405600485, 2496681174, 2659244965, 42154705, 3
28264661, 2794402597, 1658709575, 3363521597, 1566336700, 2328802979, 4035958571, 947304315, 3086967152, 2059002863, 3768563607, 3014841494, 3677339158, 1818170437, 2981664208, 3565373238, 3591266993, 4036601283, 15
40431038, 106011070, 1418552722, 4062997160, 3201346529, 4101823863, 861328203, 2696154175, 580269713, 628273369, 3358883098, 1716637111, 637264964, 1912296794, 3620533734, 98791315, 3945302754, 742672925, 35326136,
1640017426, 3780440686, 3405744207, 3057665753, 384475274, 1660026994, 2038783194, 801698642, 344371948, 408016060, 2606528644, 172307961, 1325782072, 972711596, 4081456034, 2159691645, 3687743745, 1856805740, 2564
135007, 3937135181, 2503202829, 1632117550, 4036718069, 3988638790, 2578076707, 2508120268, 3182155045, 89324487, 113903438, 3503701767, 32700933, 2910879891, 3590324085, 1743513878, 3260666055, 1053626332, 65308124
, 659528255, 1569337110, 3256796383, 956116232, 3990444024, 304262211, 2159253790, 1115861953, 3933838962, 2547040480, 3047802294, 3938063058, 4224174205, 2159680756, 810755375, 2482400966, 1231106552, 2380411828, 1
354282538, 1883243494, 3580735433, 2427068586, 3785512055, 900480325, 3821731165, 3539884527, 3281565445, 2392742875, 1183068911, 433598458, 3704004061, 1772725673, 2392432239, 1941040794, 2464961769, 370515460, 333
6289900, 3894647895, 1053553301, 1598335332, 3239810836, 1644860006, 1647641675, 3992215759, 4169949752, 2320804757, 1056733854, 2911680706, 3839420050, 2046390357, 1806386831, 3055158653, 309363189, 2957157034, 308
404156, 1477980812, 1604728646, 3756158470, 4278205796, 1148217271, 3859973958, 2746355467, 1652349044, 3130334653) resp=00
```

LOG 2 Detailed AXI read log

```
# UVM_INFO mem_scoreboard.sv(60) @ 9883045: uvm_test_top.env.mem_scb [mem_scoreboard] [MEM_SCB][WRITE][PASS] addr=2c3 data=5c40c54a
# UVM_INFO mem_scoreboard.sv(60) @ 9883055: uvm_test_top.env.mem_scb [mem_scoreboard] [MEM_SCB][WRITE][PASS] addr=2c4 data=2770fdbb
# UVM_INFO mem_scoreboard.sv(60) @ 9883065: uvm_test_top.env.mem_scb [mem_scoreboard] [MEM_SCB][WRITE][PASS] addr=2c5 data=ee437912
# UVM_INFO mem_scoreboard.sv(60) @ 9883075: uvm_test_top.env.mem_scb [mem_scoreboard] [MEM_SCB][WRITE][PASS] addr=2c6 data=fcd06ae7
# UVM_INFO mem_scoreboard.sv(60) @ 9883085: uvm_test_top.env.mem_scb [mem_scoreboard] [MEM_SCB][WRITE][PASS] addr=2c7 data=da8a06d6
# UVM_INFO mem_scoreboard.sv(60) @ 9883095: uvm_test_top.env.mem_scb [mem_scoreboard] [MEM_SCB][WRITE][PASS] addr=2c8 data=a9211bd7
# UVM_INFO mem_scoreboard.sv(60) @ 9883105: uvm_test_top.env.mem_scb [mem_scoreboard] [MEM_SCB][WRITE][PASS] addr=2c9 data=51a76aaa
# UVM_INFO mem_scoreboard.sv(60) @ 9883115: uvm_test_top.env.mem_scb [mem_scoreboard] [MEM_SCB][WRITE][PASS] addr=2ca data=3585e6c9
# UVM_INFO mem_scoreboard.sv(60) @ 9883125: uvm_test_top.env.mem_scb [mem_scoreboard] [MEM_SCB][WRITE][PASS] addr=2cb data=2883372f
# UVM_INFO mem_scoreboard.sv(60) @ 9883135: uvm_test_top.env.mem_scb [mem_scoreboard] [MEM_SCB][WRITE][PASS] addr=2cc data=b10ed906
# UVM_INFO mem_scoreboard.sv(60) @ 9883145: uvm_test_top.env.mem_scb [mem_scoreboard] [MEM_SCB][WRITE][PASS] addr=2cd data=fcbf9e75
# UVM_INFO mem_scoreboard.sv(60) @ 9883155: uvm_test_top.env.mem_scb [mem_scoreboard] [MEM_SCB][WRITE][PASS] addr=2ce data=1f4243
# UVM_INFO mem_scoreboard.sv(60) @ 9883165: uvm_test_top.env.mem_scb [mem_scoreboard] [MEM_SCB][WRITE][PASS] addr=2cf data=c2067cdf
# UVM_INFO mem_scoreboard.sv(60) @ 9883175: uvm_test_top.env.mem_scb [mem_scoreboard] [MEM_SCB][WRITE][PASS] addr=2d0 data=30d9db75
# UVM_INFO mem_scoreboard.sv(60) @ 9883185: uvm_test_top.env.mem_scb [mem_scoreboard] [MEM_SCB][WRITE][PASS] addr=2d1 data=l4abc620
# UVM_INFO axi4_monitor.sv(84) @ 9883195: uvm_test_top.env.agt.mon [axi4_monitor] [MON][WRITE] addr=09bc data='[82978122, 3054185672, 685211287, 270839458, 1173441940, 707168339, 2412853759, 2735123610, 22853811, 12
85589885, 4017472050, 761762481, 3920336423, 2461331345, 3362426444, 3154022000, 3206914374, 1391742992, 3135308494, 3672218842, 1368026756, 2738551010, 2744855263, 2891580729, 1652517135, 2203239060, 909591050, 184
7476077, 4146254561, 267430765, 2747617995, 647113074, 2010777611, 74991889, 2423714129, 853116272, 806266494, 272053362, 1352232982, 1321952519, 1622726621, 3333885237, 3791120852, 1559978244, 2315754851, 303222462
7, 1279385859, 3546885730, 1340823862, 2123860670, 4184202297, 61201817, 2379115179, 1946253686, 3055998034, 2862142357, 657239066, 2302849116, 3144124967, 3574981409, 2889141242, 2480635024, 905839113, 361795540, 2
284659789, 2070430825, 1886408968, 2893466775, 4058124761, 104655624, 674893547, 3292385015, 2119776343, 3244691775, 2212516921, 2864273998, 210715469, 1560032664, 3469043859, 1836267065, 3569552609, 232402440, 3917
313324, 3193508, 1547748682, 661716412, 3997399314, 4241517287, 3666478806, 2837519319, 1369926314, 897967817, 679687983, 2970540294, 4240416373, 2048579, 3255205087, 819583861, 346801696, 147837092] last=1 resp=00
#
# UVM_INFO axi4_scoreboard.sv(149) @ 9883195: uvm_test_top.env.scb [axi4_scoreboard] [SCB][WRITE][PASS] addr=09bc len=99 index=623 data='[82978122, 3054185672, 685211287, 270839458, 1173441940, 707168339, 2412853759
, 2735123610, 22853811, 1285589885, 4017472050, 761762481, 3920336423, 2461331345, 3362426444, 3154022000, 3206914374, 1391742992, 3135308494, 3672218842, 1368026756, 2738551010, 2744855263, 2891580729, 1652517135,
2203239060, 909591050, 1847476077, 4146254561, 267430765, 2747617995, 647113074, 2010777611, 74991889, 2423714129, 853116272, 806266494, 272053362, 1352232982, 1321952519, 1622726621, 3333885237, 3791120852, 1559978
244, 2315754851, 3032224627, 1279385859, 3546885730, 1340823862, 2123860670, 4184202297, 61201817, 2379115179, 1946253686, 3055998034, 2862142357, 657239066, 2302849116, 3144124967, 3574981409, 2889141242, 248063502
4, 905839113, 361795540, 2284659789, 2070430825, 1886408968, 2893466775, 4058124761, 104655624, 674893547, 3292385015, 2119776343, 3244691775, 2212516921, 2864273998, 210715469, 1560032664, 3469043859, 1836267065, 3
569552609, 232402440, 3917313324, 3193508, 1547748682, 661716412, 3997399314, 4241517287, 3666478806, 2837519319, 1369926314, 897967817, 679687983, 2970540294, 4240416373, 2048579, 3255205087, 819583861, 346801696,
147837092] resp=00
# UVM_INFO mem_scoreboard.sv(60) @ 9883195: uvm_test_top.env.mem_scb [mem_scoreboard] [MEM_SCB][WRITE][PASS] addr=2d2 data=8cfd0a4
# UVM_INFO mem_scoreboard.sv(66) @ 9883255: uvm_test_top.env.mem_scb [mem_scoreboard] [MEM_SCB][READ][PASS] addr=15b expected=1622d1f3 actual=1622d1f3
# UVM_INFO axi4_driver.sv(142) @ 9883265: uvm_test_top.env.agt.drv [axi4_driver] [DRV][READ] beat=0 data=371380723 RLAST=0
# UVM_INFO mem_scoreboard.sv(66) @ 9883285: uvm_test_top.env.mem_scb [mem_scoreboard] [MEM_SCB][READ][PASS] addr=15c expected=90e358e4 actual=90e358e4
# UVM_INFO axi4_driver.sv(142) @ 9883295: uvm_test_top.env.agt.drv [axi4_driver] [DRV][READ] beat=1 data=2430818532 RLAST=0
# UVM_INFO mem_scoreboard.sv(66) @ 9883315: uvm_test_top.env.mem_scb [mem_scoreboard] [MEM_SCB][READ][PASS] addr=15d expected=773d1bfd actual=773d1bfd
# UVM_INFO axi4_driver.sv(142) @ 9883325: uvm_test_top.env.agt.drv [axi4_driver] [DRV][READ] beat=2 data=2000493565 RLAST=0
# UVM_INFO mem_scoreboard.sv(66) @ 9883345: uvm_test_top.env.mem_scb [mem_scoreboard] [MEM_SCB][READ][PASS] addr=15e expected=840aa183 actual=840aa183
# UVM_INFO axi4_driver.sv(142) @ 9883355: uvm_test_top.env.agt.drv [axi4_driver] [DRV][READ] beat=3 data=2215289219 RLAST=0
# UVM_INFO mem_scoreboard.sv(66) @ 9883375: uvm_test_top.env.mem_scb [mem_scoreboard] [MEM_SCB][READ][PASS] addr=15f expected=65falb0a actual=65falb0a
# UVM_INFO axi4_driver.sv(142) @ 9883385: uvm_test_top.env.agt.drv [axi4_driver] [DRV][READ] beat=4 data=1710889738 RLAST=0
# UVM_INFO mem_scoreboard.sv(66) @ 9883405: uvm_test_top.env.mem_scb [mem_scoreboard] [MEM_SCB][READ][PASS] addr=160 expected=lee5fddc actual=lee5fddc
# UVM_INFO axi4_driver.sv(142) @ 9883415: uvm_test_top.env.agt.drv [axi4_driver] [DRV][READ] beat=5 data=518389197 RLAST=0
# UVM_INFO mem_scoreboard.sv(66) @ 9883435: uvm_test_top.env.mem_scb [mem_scoreboard] [MEM_SCB][READ][PASS] addr=161 expected=e7ffed05 actual=e7ffed05
# UVM_INFO axi4_driver.sv(142) @ 9883445: uvm_test_top.env.agt.drv [axi4_driver] [DRV][READ] beat=6 data=3892309253 RLAST=0
# UVM_INFO mem_scoreboard.sv(66) @ 9883465: uvm_test_top.env.mem_scb [mem_scoreboard] [MEM_SCB][READ][PASS] addr=162 expected=5787ed6e actual=5787ed6e
# UVM_INFO axi4_driver.sv(142) @ 9883475: uvm_test_top.env.agt.drv [axi4_driver] [DRV][READ] beat=7 data=1468525934 RLAST=0
# UVM_INFO mem_scoreboard.sv(66) @ 9883495: uvm_test_top.env.mem_scb [mem_scoreboard] [MEM_SCB][READ][PASS] addr=163 expected=eb6dd8df actual=eb6dd8df
# UVM_INFO axi4_driver.sv(142) @ 9883505: uvm_test_top.env.agt.drv [axi4_driver] [DRV][READ] beat=8 data=3949844703 RLAST=0
# UVM_INFO mem_scoreboard.sv(66) @ 9883525: uvm_test_top.env.mem_scb [mem_scoreboard] [MEM_SCB][READ][PASS] addr=164 expected=7aa648df actual=7aa648df
# UVM_INFO axi4_driver.sv(142) @ 9883535: uvm_test_top.env.agt.drv [axi4_driver] [DRV][READ] beat=9 data=2057717983 RLAST=0
```

READ WAVEFORM



WRITE WAVEFORM

Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_instances	Get_inst_coverage	Comment
/axi4_coverage_pk...		100.00%							
TYPE cg		100.00%	100	100.00...		✓			auto(1)
CVP cg::c_...		100.00%	100	100.00...		✓			
bin b1		246	1	100.00...		✓			
bin b2		239	1	100.00...		✓			
bin b3		328	1	100.00...		✓			
bin b4		268	1	100.00...		✓			
bin b5		272	1	100.00...		✓			
bin b6		294	1	100.00...		✓			
bin b7		215	1	100.00...		✓			
bin b8		138	1	100.00...		✓			
CVP cg::c_...		100.00%	100	100.00...		✓			
bin b1		287	1	100.00...		✓			
bin b2		291	1	100.00...		✓			
bin b3		294	1	100.00...		✓			
bin b4		280	1	100.00...		✓			
bin b5		274	1	100.00...		✓			
bin b6		277	1	100.00...		✓			
bin b7		228	1	100.00...		✓			
bin b8		69	1	100.00...		✓			
CVP cg::c_...		100.00%	100	100.00...		✓			
bin b0		12	1	100.00...		✓			
bin b1		576	1	100.00...		✓			
bin b2		475	1	100.00...		✓			
bin b3		474	1	100.00...		✓			
bin b4		463	1	100.00...		✓			
CVP cg::c_...		100.00%	100	100.00...		✓			
bin b0		7	1	100.00...		✓			
bin b1		538	1	100.00...		✓			
bin b2		568	1	100.00...		✓			
bin b3		444	1	100.00...		✓			
bin b4		443	1	100.00...		✓			
CVP cg::c_...		100.00%	100	100.00...		✓			
ignore...		0	-	-		✓			
bin oka...		2000	1	100.00...		✓			
CVP cg::c_...		100.00%	100	100.00...		✓			
ignore...		0	-	-		✓			
bin oka...		2000	1	100.00...		✓			
CVP cg::c_...		100.00%	100	100.00...		✓			
bin wor...		2000	1	100.00...		✓			
CVP cg::c_...		100.00%	100	100.00...		✓			
bin wor...		2000	1	100.00...		✓			

FUNCTIONAL COVERAGE

Statement Coverage:				
Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Statements	82	82	0	100.00%

STATEMENT COVERAGE

==== Instance: /top/dut				
==== Design Unit: work.axi4				
=====				
▼ Branch Coverage:				
Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Branches	22	22	0	100.00%
=====Branch Details=====				

BRANCH COVERAGE

=====Condition Details=====				
▼ Condition Coverage for instance /top/dut --				
File axi4.v				
-----Focused Condition View-----				
Line	259	Item	1	(read_burst_cnt > 0)
Condition totals: 1 of 1 input term covered = 100.00%				
▼ Expression Coverage:				
Enabled Coverage	Bins	Covered	Misses	Coverage
-----	----	----	-----	-----
Expressions	1	1	0	100.00%

CONDITION COVERAGE

=====Expression Details=====				
Expression Coverage for instance /top/dut --				
File axi4.v				
-----Focused Expression View-----				
Line	254	Item	1	(read_burst_cnt == 0)
Expression totals: 1 of 1 input term covered = 100.00%				
FSM Coverage:				
Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
FSM States	8	8	0	100.00%
FSM Transitions	9	9	0	100.00%

EXPRESSION COVERAGE

Summary	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
FSM States	4	4	0	100.00%
FSM Transitions	5	5	0	100.00%

FSM COVERAGE

Toggle Coverage:	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Toggles	470	470	0	100.00%

TOGGLE COVERAGE

1 Coverage Report by instance with details

8 Assertion Coverage:

[illegible]

24

ASSERTION COVERAGE

```
1  vlog -f files.txt +cover -covercells
2
3  vsim -coverage -assertdebug -voptargs=+acc work.top -cover
4  do exclude.do
5  coverage save axi4_coverage.ucdb
6  do wave.do
7  run -all
8  coverage report -details -output cov_report.txt
9
```

RUN.DO